# 読み上げ台本(日本語) — WireGuard for Apache NuttX (Community Over Code, Glasgow)

本番は英語で話す前提(デッキは英語、coc-glasgow-outline.md より "English, ~20-30 min")。この日本語版は英語版と 1対1で対応する**リハーサル・理解確認用**の台本 — 内容を体に入れる、または国内で 日本語版として話す場合のベース。英語版が更新されたら、この対応も崩れるので 両方直すこと。

**40枚・デモ90秒を含めて合計約31分(30:36)。** 2026-10-12、発表者の改善メモを受けて簡素化: 第4部の落とし穴は「6つのバグの一覧表」1枚 + 詳しく話す2件(ping は通るのに TCP が死ぬ / 50ms ポーリングで10倍遅い)にし、落とし穴1(detach された pthread)と4(実機のスタック)は 一覧表とノートへ。第5部は「なぜカーネルへ」の意図を明確にし、設計は図1枚に、検証は FLAT(実機)/ KERNEL(QEMU)/ PROTECTED(QEMU)を分けたカード3枚に、「ping が見逃すもの」と タイムスタンプは短く。第4・5部に GitHub への案内(QRコード付き)。第7部は運用性と移植性の 2枚を削り(要点はまとめのノートへ)、「What I learned」を1枚追加。2026-10-10、目次を7部(概要と背景 / 課題とアイデア / 計画 / 第1版 / 第2版 / ライブデモ / まとめ)に組み直し、第1版の動画と謝辞の2枚を追加。2026-10-10、ノートに小ネタ4つ(14・20・31・34、斜体で出典を付記)。知らない人には普通の文として聞こえるものだけで、外しても流れは変わらない。 時間は累積で、英語版と同じ値(英語の読み上げ速度で計算)。

**重心はスライド29〜31(検証)** — 約4分をこの3枚が占める。査読者が実際に確認してくる主張はここに集中している。**スライドも読み上げも短くしてある** (2026-10-12。細部は Q&A と GitHub へ)ので、**主張と注記(実機は FLAT / KERNEL と PROTECTED は QEMU のみ / sim、実機の以前の版と 2026-10-12 の ESP32-S3 での提出シリーズそのものの実行の区別、PROTECTED の wg スタック不足とその修正、4 CPU は1回の スクリプト実行、PROTECTED は実機では未確認、75秒見ていて)はこれ以上削らない。**削るならこの3枚の前か後。29→30→31は意味的にも一体: 29「どこで動くか」→30「動いたことは何を証明しないか」→ 31「唯一、判断で答えた場所」。分割すると論旨が崩れる。

**スライド2と4〜15(About Me、Overview〜Architecture。経緯と比較の2枚を含む)も削らない。** NuttXとは何か、どこで 使われているか、Sonyでの歴史、自分がどこで出会ったか、そこからWireGuardに至った 理由 — この因果の連鎖が、後半すべての前提になる。**スライド37(What I learned)も残す** — 発表者自身のメッセージ。

**時間が足りないときに削る順番:**

1. スライド36(Giving back: where it stands — 還元の現状報告は本編の主張を支える**おまけ**で、背骨ではない。真っ先に削ってよい。削る場合は Q&A で聞かれたら答える程度に留める)
2. スライド24(第1版の動画の静止画 — ライブデモが同じ種類のものを見せる)
3. スライド23(落とし穴 3/6、50ms ポーリング — 1行の教訓は一覧表にすでにある)

ライブデモが止まったら、すぐローカル録画へ切り替える。**目次に戻るスライド(4・11・16・19・25・32・35)は各10秒程度の一呼吸** — 読み上げる ものではなく、指し示して一拍置くだけ。本当に時間がなければ無言で次へ送ってもよい (時間短縮にはほぼ寄与しないので、切るのはここではなく上の3枚から)。

**本番前に確認:** (1) 合計約31分は30分枠を少し超えている。本編は約28分を目標にリハーサルし、上の削る順番を本番前に決めておく。(2) GitHubスター数(FreeRTOS/Zephyr/NuttXの比較スライド)は変動するので、本番直前に実数を再確認するか、スライド上の日付表記をそのまま残す。

---

## 1 — タイトル (0:00 → 0:50)

**画面:** "WireGuard for Apache NuttX"。ESP32-S3 と SPRESENSE が実 Wi-Fi でトンネル / 実際の WireGuard ピア (Linux kernel, Windows クライアント) / QEMU 上のカーネルビルドで動作 (BUILD_KERNEL) / NuttX 13.0.1・master・12.7.0 が同一ソース。

**話す:**

- 今日は WireGuard を Apache NuttX に持ち込む話をします。
- これは `wg0` というネットワークデバイスとして実装した WireGuard VPN です。ESP32-S3 と SPRESENSE、両方とも実 Wi-Fi 経由でトンネルが立っています。ピアは 常に本物の WireGuard 実装 — Linux カーネルモジュールと公式 Windows クライアント — です。apps版は同じソースで NuttX の3バージョンに対応します。話の後半では、新しいドライバが QEMU 上の KERNEL / PROTECTED ビルドで、そして両ボードの通常の FLAT ビルドで動きます。ここが今日の、より難しく、より 面白い半分です。
- ただ、この話の主眼は「動いた」ことではありません。「動く」が何を隠していたか、そしてカーネルに押し込んだことで何が表に出たか、という話です。

## 2 — 自己紹介 (0:50 → 1:44)

**画面:** 見出し「From Japan: I work at Sony, and I build things as a hobby.」。上: 本人写真・ 名前の1行・SNS の狐アイコン。下の左: 日本地図(① 名古屋 生まれ → ② 仙台 大学 → ③ 川崎 現在)。下の右: Hobby engineer の写真5枚(Space: モデルロケット / Semiconductors: ウェハ上の試作チップ / DNA nanotech: cryo-TEM 像 / Robots: ハッカソンのヒューマノイド / Linux: LinuC のペンギン)と、活動のチップ3つ。経歴の流れ(分子ロボティクス → … → NuttX)は画面にはなく、口頭だけ。

**話す:**

- 簡単に自己紹介します — どこから来たか、という話です。
- Sony Semiconductor Solutions で edge AI エンジニアをしていて、application 側から Apache NuttX に関わっています。日本から来ました: 名古屋生まれ、大学は仙台、 今は川崎に住んでいます。仕事の外ではホビーエンジニアで、宇宙・半導体・ ロボットまわりのものを作っています。大学では DNA でナノ構造を作り、Linux も大好きです。SNS では、この写真の狐が私です。ここに至る道のりは、分子ロボット工学の世界大会、コンピュータビジョンと AI システムの仕事、edge AI カメラ、そして SPRESENSE 上の embedded systems を通っています — NuttX と WireGuard は、その最新の到達点であって、 出発点ではありません。
- ここではっきりさせておきたいことがあります: 私は NuttX の kernel developer として 始めて、VPN を追加しようと思い立ったわけではありません。私は application 側・ deployment 側から NuttX に繰り返し出会い続けていて、ある時、その立場からは 解決できない課題に行き当たりました。それが今日の話です。

## 3 — 目次 (1:44 → 2:23)

**画面:** 7つの部。見出しは短い名詞句、その下に中身のキーワードを1行、右に所要時間 — Overview & background(~5分)/ The problem and the idea(~4分)/ The plan(~2分)/ Version 1: the app — 6つのバグ、うち2つを詳しく(~4分)/ Version 2: the kernel driver — なぜドライバか、 NuttX のバグ、試験(~7分)/ Live demo(~3分)/ Summary & status — 今の状態、学んだこと、謝辞(~4分)。

**話す:**

- 今日の話の地図です。
- 7つの部があります。まず概要と背景: NuttX とは何か、そして この取り組みの始まり。次に、解きたかった課題と、そのアイデア。3つ目が計画。 4つ目は第1版、アプリとしての実装: 最初のトンネル、それが隠していたバグ、 そして最初のデモ。5つ目は第2版、カーネルドライバ: なぜ移したか、NuttX 自身の バグ、そして試験。6つ目はスタックちゃんでのライブデモ。最後に、作業が今どこに あるか、学んだこと、そしてまとめです。
- 全部で約30分、その後質疑応答です。
- この後、部が変わるたびにこの目次に戻ってきて、今どこにいるかを示します。

## 4 — 目次 ①Overview & background (2:23 → 2:33)

**画面:** 目次と同じリスト。①Overview & background だけハイライト、他は薄い。

**話す(一言):** では、第1部の概要と背景から。

**間:** ハイライトされた行を指して一拍。

## 5 — プロジェクト概要 (2:33 → 3:23)

**画面:** `wg0` netdev の図 — アプリケーション/NSH、NuttX ネットワークスタック、 `eth0`/`wlan0` の隣に `wg0`、その下に UDP ソケット、インターネット、ピア。

**話す:**

- 全体像を1枚の絵にするとこうなります。
- Apache NuttX は独自の TCP/IP スタックを持つ POSIX 準拠の RTOS です。VPN は ありませんでした。このプロジェクトは `wg0` を仮想ネットワークインターフェース として追加します — アプリケーションからは普通のインターフェースに見え、その下では暗号化された伝送路が UDP ソケットです。反対側のピアは本物の WireGuard エンドポイント — Linux、Windows、あるいは別のゲートウェイです。

## 6 — 経緯: 2月の思いつきから10月の本番まで (3:23 → 4:15)

**画面:** 2月〜10月の月の横軸。上に出来事 — 2月 中旬 GSoC のテーマ一覧を見て調べ始める / 3月 上旬 計画を提案 / 3月20日 Alan さんからこのカンファレンスを聞き、講演を申し込む / 4月 上旬 GSoC に応募 / 5月 上旬 講演は採択、GSoC は不採択 — それでも続ける / 10月 グラスゴー。下に開発 — 6〜7月 少しずつ / 8月の休み(お盆)に開発の大半 / 9月〜 FLAT のアプリからカーネルドライバへ。

**話す:**

- このプロジェクトがどう進んだかを、月ごとに見せます。
- 2月の中旬に GSoC のテーマ一覧を見て、調べ始めました。3月の初めに計画を Issue に まとめて提案しました。3月20日に Alan さんからこのカンファレンスのことを聞いて、 講演を申し込みました。4月の初めに GSoC に応募しました。
- 5月の初めに2つの返事が来ました。講演は採択、GSoC は不採択。それでも作ることに しました。
- 6月からは少しずつ進め、まとまった開発は8月の休みにやりました。9月には FLAT の アプリからカーネルドライバに作り直しました — それがこの話の後半です。

## 7 — Background: NuttX とは何か (4:15 → 5:07)

**画面:** ソフトウェアスタック図(Applications / POSIX・ANSI・BSD socket API / Networking・VFS・filesystems・device drivers・NSH shell / NuttX kernel / MCU・MPU)。下にタグ: RTOS, POSIX-oriented, BSD sockets, Filesystem/VFS, NSH shell, Native networking, Device drivers, Apache Software Foundation TLP。

**話す:**

- 本題に入る前に、NuttX そのものを説明します。
- NuttX は、標準準拠を第一に置いたリアルタイム OS です。POSIX・ANSI API に準拠し、ネットワークは BSD socket で話し、本物のファイルシステムと VFS 層を持ち、NSH というシェルがあり、ネイティブなネットワーキングとデバイスドライバのモデルを 持ちます — これら全部を、8 ビットから 64 ビットのマイクロコントローラで動く サイズまで縮めています。NuttX は Apache Software Foundation の top-level project でもあり、kernel も applications も documentation も、contributor community に よって公開の場で開発されています。
- ここで言いたいことは一つに絞ります: NuttX は小さな RTOS でありながら、Unix 的な プログラミングモデルを持っている。この組み合わせは珍しく、この後出てくる socket、netdev、POSIX の形をした porting が可能なのは、すべてこれのおかげです。

## 8 — Positioning: NuttX の立ち位置と実際の採用例 (5:07 → 6:17)

**画面:** 左に RTOS 比較表(FreeRTOS / Zephyr / NuttX、design center と typical use)、GitHub スター数(FreeRTOS/FreeRTOS 7.8k・zephyr 16.5k・apache/nuttx 4.1k、「スターは可視性であって導入量ではない」の注記)。右に数字カード(15+アーキテクチャ、300+ボード、1500+コンフィグ、Sony / PX4 drones / Xiaomi OpenVela 1000+SKU / 日本の2024年月面ミッション)。

**話す:**

- 30秒だけ、NuttX がどこに位置するか、そしておもちゃのプロジェクトではないか、という話をします。
- 3つとも有力な RTOS で、重心が違います。FreeRTOS は意図的に小さいリアルタイム カーネルで、MCU ファームウェアや制御、IoT 向け。Zephyr は接続デバイス向けの 統合プラットフォームで、ウェアラブルや産業用・接続 IoT 向け。NuttX は Unix 的な 方 — ネットワーキング、ファイルシステム、ドライバがあり、ドローンやセンシングで 使われます。優劣表ではなく、形の違いです。GitHub のスター数で言えば、NuttX は 3つの中で一番小さい、とはっきり言います。スター数は開発者からの可視性であって、実際の導入量ではありません。
- 導入面は実在します: 15 以上のアーキテクチャ、300 以上のボード、1500 以上の コンフィグテンプレートが upstream にあります。実運用では Sony のデバイス、PX4 の フライトコントローラ、Xiaomi OpenVela の 1000 以上の SKU、そして日本の 2024 年の 月面ミッションでも飛んでいます。大きなエコシステムの隣では小さいですが、おもちゃではありません。

## 9 — History: Sony と NuttX の歴史 (6:17 → 7:19)

**画面:** 縦タイムライン — 2015 Sony audio products / 2018–19 SPRESENSE・CXD56xx (OSS+ELC Europe 2019 で発表) / 2020 upstream collaboration(NuttX Online Workshop、「A Journey from Fork to Mainline」) / 2020s sensing & edge devices。

**話す:**

- これは Sony の NuttX との歴史であって、まだ私の話ではありません — そして edge AI より長いです。
- Sony は 2015 年ごろから、NuttX を使った音声製品を出荷しています。数年後に登場した SPRESENSE とその CXD56xx チップは、音声、GNSS、カメラ、マルチコアのサポートを NuttX に持ち込み、2019 年に Sony の Masayuki Ishikawa 氏が Open Source Summit + Embedded Linux Conference Europe でその成果を発表しています。そこから、Alin Jerpelea 氏を含む Sony のエンジニアたちが、upstream への大きな貢献者に なっていきます — 後に NuttX の project management committee の議長も務める 人物です。彼の 2020 年の講演は文字通り「A Journey from Fork to Mainline (fork から mainline への旅)」という題でした。
- 一人のエンジニアがいたから Sony が NuttX を採用した、という因果関係は主張しません — それを裏付ける一次情報がないからです。公開されている講演や文書から言えることは もっとシンプルです: Sony の NuttX 利用は edge AI より何年も前からあり、カメラや GNSS、センシングの仕事へと時間をかけて広がっていった。これが、私自身の経験が 立っている土台です。

## 10 — Experience: SPRESENSE と AITRIOS での私の経験 (7:19 → 8:19)

**画面:** SPRESENSE と AITRIOS / edge AI カメラの実写真。下に "application development → system integration → deployment & operation" の フロー。

**話す:**

- 今のが Sony の歴史です。これは私自身の話です — 同じハードウェアファミリーですが、立ち位置が違います。
- SPRESENSE では、NuttX SDK を使って、センシングとカメラ、GNSS・LTE まわりの プロトタイピングと embedded 開発をしています。AITRIOS の edge AI カメラでは、ESP32 と NuttX で構築されたカメラを、ベンチの上ではなく実際に現場に設置して 動かす、という仕事です。
- 共通しているのは、私が NuttX に近づく方向です: application 開発、そして system integration、そして deployment・運用。これまで、そのために NuttX の ドライバを自分で書く必要は一度もありませんでした — 今回のプロジェクトまでは。そして私をそこへ連れてきた課題は、「NuttX をどう動かすか」ではありませんでした。「デプロイされた後の NuttX デバイスに、どうやって届くか」でした。

## 11 — 目次 ②The problem and the idea (8:19 → 8:29)

**画面:** 目次と同じリスト。②The problem and the idea だけハイライト、他は薄い。

**話す(一言):** 背景は以上です。第2部: 解きたかった課題と、そのアイデア。

**間:** ハイライトされた行を指して一拍。

## 12 — Problem: 遠隔アクセスの隙間 (8:29 → 9:22)

**画面:** Engineer → Internet → Router/network → Wi-Fi → NuttX device の図。WPA2/WPA3 が守る範囲(Wi-Fi の一区間のみ)と、必要な認証済み end-to-end パスの 対比。下に3枚のカード: open it to the Internet / make your own protocol / use a vendor's cloud。

**話す:**

- ここからは課題そのものを、率直に話します。

- Wi-Fi のセキュリティは本物ですが、守っているのは一区間だけです: デバイスと アクセスポイントの間の無線リンク。それより手前 — 技術者、インターネット、ルーター — は、WPA2 や WPA3 が約束できる範囲の外です。私が必要としていたのは、経路全体 — 技術者からデバイスまで — の認証と暗号化で、最後の数メートルだけ ではありません。
- それがないと、実務的な選択肢はどれも今より悪くなります: デバイスにグローバル IP を公開して祈る、独自のセキュアプロトコルを発明して保守し続ける、あるいは 接続をベンダーのクラウドに委ねる。同じ隙間は、edge AI や産業用 IoT、遠隔 インフラ全般にも現れます — このプロジェクト固有の話ではありません。

## 13 — 他の手段との比較 (9:22 → 10:44)

**画面:** 7つの手段(インターネットにポートを開ける / クラウド API への HTTPS / MQTT ブローカー / リバーストンネル・リレー / ルーターに VPN / キャリアの閉域網 / 機器に VPN)を、誰から接続するか・任意のサービスに届くか・間にサーバーが要るか・ どこまで守られるか・NuttX に今あるもの、で比べた表。最後の行(機器に VPN)が今回の話。

**話す:**

- WireGuard を選ぶ前に、機器に届くための一般的な方法を比べました。
- ポートを開けるのは簡単ですが、インターネット全体から届いてしまいます。HTTPS と MQTT は多くの IoT 製品が使っている方法です。機器からサーバーへ接続しに行くので NAT は問題にならず、NuttX にも両方のクライアントがすでにあります。ただし届くのは 自分で設計したメッセージだけで、機器にログインしたり Web ページを開いたりは できません。暗号化もブローカーやクラウドで終わります。
- リバーストンネルは、リレーサーバー経由でポートを1つ通します。ルーターの VPN は ルーターが自分のものなら良い方法ですが、LAN の最後の一区間は守られず、顧客の 現場ではルーターは普通こちらのものではありません。キャリアの閉域網はセルラー 回線だけで、契約が要ります。
- 機器の上の VPN だけが全部を満たします: 任意の IP サービス、間にサーバーなし、機器までの暗号化。データのやり取りには今でも MQTT や HTTPS が正しい道具で、これは保守のためのアクセスです。

**Tailscale や ZeroTier について聞かれたら:** これらも機器上の VPN だが、今のところ マイコンより大きな OS が要ることが多い(スライドの脚注)。

## 14 — Solution: なぜ WireGuard なのか (10:44 → 11:37)

**画面:** Engineer/Linux/Windows ↔ WireGuard トンネル ↔ NuttX device の図。下に3点: modern IP-layer VPN / UDP transport, public-key peers / already everywhere。

**話す:**

- その課題に対して、なぜ WireGuard なのか、です。
- WireGuard はモダンな IP 層の VPN です: 設計上小さくシンプルで、IPsec や OpenVPN に比べてコードサイズがごく一部です — これはマイクロコントローラの 上では大きな意味を持ちます。素の UDP の上で動き、ピアは証明書や交渉された セッションではなく公開鍵で識別されます。正しい鍵を持たないパケットには 一切返事をしません — 外から見れば、ただのレンガの壁です *(9¾ 番線へのオマージュ。知らなくても普通の比喩として通じる)*。そして重要なのは、特殊なものでは ないということです: 何年も前から Linux カーネルに mainline で入っていて、公式の Windows クライアントがあり、トンネルの反対側は既存のどんな WireGuard ピアでも構いません。
- つまり選んだのは「製品」ではなく、「独自プロトコルを発明する、あるいは ベンダーのクラウドに頼る代わりに、標準プロトコルを採用する」という選択です。これが、この特定の課題に WireGuard が合う理由です。

## 15 — Architecture: NuttX ネットワークスタック上の WireGuard (11:37 → 12:25)

**画面:** Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0/usrsock・物理ネットワーク、の層状図。左右に TX/RX 両方向の矢印。

**話す:**

- 概要図より一段技術的になります。ここから実装の話が始まります。
- 送信方向では、アプリケーションから TCP または UDP・IP を通って降りてきた パケットが、他の netdev と同じように `wg0` に届き、WireGuard によって 暗号化され、ボード上では `wlan0` を通る、あるいは Wi-Fi そのものがソケット ベースの周辺機器である `usrsock` を通る UDP データグラムとして出ていきます。受信方向はこの逆です: UDP データグラムが届き、WireGuard が復号し、普通の インターフェースに届いたかのように `wg0` へ渡されます。
- ここから先はすべて、この真ん中の部分 — `wg0` と暗号化のステップ — を、上下の層に触れずに作る話です。

## 16 — 目次 ③The plan (12:25 → 12:35)

**画面:** 目次と同じリスト。③The plan だけハイライト、他は薄い。

**話す(一言):** 第3部: 計画。既存の WireGuard をどう再利用するか。

**間:** ハイライトされた行を指して一拍。

## 17 — Porting Strategy: wireguard-lwip を lwIP なしで再利用する (12:35 → 13:39)

**画面:** 3段カード — Protocol & crypto(KEEP、緑)/ Platform hooks(ADAPT、金、 `wireguard-platform.h` の4関数)/ lwIP network glue(REPLACE、赤)。

**話す:**

- ゼロから暗号スタックを書くのは間違いです。なので `smartalock/wireguard-lwip` という既存実装から始めました、その理由です。
- `wireguard-lwip` はきれいに3層に分かれています。プロトコルと暗号 — ハンドシェイク、トランスポート、BLAKE2s、ChaCha20-Poly1305、Curve25519 — は OS 依存のない移植可能な C です。暗号は無変更のまま使いました。プロトコル本体は、後のレビューで小さな修正が2つ必要になり、PATCHES.md に書いてあります。OS 依存の 振る舞いはすべて、すでに1つのヘッダー、`wireguard-platform.h` の裏に、4つの関数として隔離されていました。置き換えが必要だったのは lwIP の ネットワークグルーの部分だけです — lwIP の netif を登録し、lwIP のバッファを 管理する部分です。NuttX には lwIP がそもそも存在しないからです。
- 何も書き直さずに済んだのは運ではなく、きれいなプラットフォーム境界が もたらすものです。その境界があったからこそ、最初の移植作業は書き直し ではなく差し替えで済み、作業量そのものが桁違いに小さくなりました。

## 18 — Design: lwIP を NuttX netdev へ写像する (13:39 → 14:08)

**画面:** lwIP → NuttX の対応表(`netif` → `net_driver_s`、`pbuf` → `iob`、 `netif_add()` → `netdev_register()`、コールバック → `devif_poll()`)。

**話す:**

- 移植の中で最も重要な判断です。
- lwIP を NuttX に引きずり込むのではなく、抽象化を NuttX 自身のものに写像 しました。`netif` は `net_driver_s` に、`pbuf` は `iob` に、`netif_add` は `netdev_register` に、送信コールバックは NuttX の `devif_poll` になります。 `wg0` は TUN 型の netdev として登録され、その下の「配線」は UDP ソケットです。

## 19 — 目次 ④Version 1: the app (14:08 → 14:18)

**画面:** 目次と同じリスト。④Version 1: the app だけハイライト、他は薄い。

**話す(一言):** 第4部: 第1版、アプリとしての実装。動いた、そしてバグが出てきました。

**間:** ハイライトされた行を指して一拍。

## 20 — Result: 動くトンネルは始まりにすぎない (14:18 → 15:03)

**画面:** 表(Protocol & crypto: 暗号はバイト単位で同一、プロトコル本体は小さな修正2つ(PATCHES.md) / OS hooks: 4関数 / Network glue: 実際の作業)。`wg genkey / set / setconf` が実行時に可能、という claim。

**話す:**

- 最初のバージョンの結果です。
- プロトコルと暗号処理、3000行以上を持ってきました。暗号はそのままです。プロトコル本体はレビューの後で小さな修正が2つ必要でした — そのうち1つは wireguard-lwip 自身のバグで、レート制限が一度も効いていませんでした。4つの OS フックは小さなものです。本当の作業はネットワークの つなぎ込みでした。そして動きました: 鍵の生成、ピアの設定、設定ファイルの 読み込み、すべて実行時に、再ビルドなしで。
- ここまでは成功譚に見えます — でも、お姫様は別の城にいました *(マリオの「Your princess is in another castle!」)*。このスライドの本当の役割は、そこです —「動いた」ように見えるところまでが実は一番安く手に入る到達点で、この後の話はほとんど、その「動いた」が何を隠していたか、です。

## 21 — Where it was hard: 6つのバグ (15:03 → 16:12)

**画面:** 見出し「The tunnel worked fast. Six bugs took most of the time.」。6つのバグの一覧表 (バグ / 出た場所 / 教訓): 1 `wg` コマンドが終わると受信スレッドが止まる / 2 ping は通るのに TCP が死ぬ (in detail) / 3「実害なし」が10倍 (in detail) / 4 スタック94%、設定保存でボードが固まる、実機だけ / 5 NuttX 自身の ChaCha20-Poly1305(part 5) / 6 3KB のカーネルスタックに 6KB の配列(part 5)。GitHub への案内(QRコード付き)。Takeaway: 多くは通る試験の陰に隠れていた。

**話す:**

- ここからは、苦労したところです。
- 最初のトンネルはすぐに動きました。時間の大半を使ったのは6つのバグです。apps 版で4つ、カーネルに移してからさらに2つ。多くは、ハンドシェイクや ping のような通る試験の陰に隠れていたので、最初は違う場所を探していました。
- バグ1: 受信スレッドは detach 付きで作られていて、それを起動した `wg` コマンドが 終わると、一緒に止まりました。detach が決めるのはメモリがいつ解放されるかだけで、スレッドがどれだけ生きるかではありません。カーネル版でスレ

ッドをドライバが 持っているのは、これが理由です。バグ4: 実機だけが見せてくれました。ボード上で、受信タスクのスタックは94%使われていて、残りは176バイトだけでした。sim はスタックが 大きいので、一度も見えませんでした。これは覚えておいてください。
- このうち2つを詳しく話します。その教訓は誰にとっても役に立つからです。バグ5と6は 第5部で出てきます。6つすべてのログと修正は GitHub のリポジトリにあります。

**バグ1と4には、もう専用のスライドがない。** 話は上のノートにある。詳しい版(バグ1の `SO_RCVTIMEO` の回り道を含む)は Q&A 資料にある。

## 22 — 落とし穴 2/6: ping は通るのに TCP が死ぬ (16:12 → 17:07) *(お気に入り)*

**画面:** Takeaway は「If only some tasks fail, look at what each task owns. Here, it was the file descriptor.」

**話す:**

- このプロジェクトで一番好きなバグです。
- 同じトンネルの上で、ping は完璧に通り、TCP の 3-way ハンドシェイクすら 完了する — それなのに接続は死にました。送信経路を計装すると `EBADF`、不正なファイルディスクリプタが出ていました。成功していた送信はすべて 受信タスク自身のコンテキストで行われたものでした — ハンドシェイクの 応答、キープアライブ、ping の応答、SYN-ACK。他のタスクからの送信はすべて 失敗していました。
- 原因: NuttX ではファイルディスクリプタはタスクグループ単位にスコープされ ていて、システム全体でもグローバルでもありません。修正は fd を経由する こと自体をやめ、NuttX の内部ソケット API を使うことでした — これは `struct socket` に対して動作する、ただのメモリなので、どのタスクからでも 使えます。カーネル版がソケットを `struct socket` として保持しているのは、まさにこの理由です。

## 23 — 落とし穴 3/6:「実害なし」が実は10倍 (17:07 → 17:48) *(削る候補3)*

**話す:**

- 3つ目: 測定しなかった前提。
- fd を使わない修正は `poll` も壊しました。`poll` は fd ベースだからです。応急処置として 50ミリ秒ごとにソケットをポーリングしました。コストは アイドル時に CPU を起こすことだけだと思っていました。間違いでした。 届いたパケットはすべて最大 50ミリ秒待たされ、それだけでスループットの 上限になっていました — 本来より約10倍遅い。実害なしだと思っていたものが、実は10倍でした。本当の修正は正しいブロッキング待ち — `psock_poll` への コールバックとセマフォです。

## 24 — 第1版のデモ: 最初の動画 (17:48 → 18:14) *(削る候補2)*

**画面:** 左に動画の1コマ(公式 Windows WireGuard クライアントと、ESP32-S3 上の NuttX への telnet セッションが並んだ画面)。右に手順 — telnet 10.10.0.2 / uname -a、ifconfig / webserver & / ブラウザで http://10.10.0.2/ — と条件(ESP32-S3 DevKit、apps 版 (FLAT)、USB は抜いてある、相手は 公式 Windows クライアント)、小さく youtu.be/1kyX2av5WG4。

**話す:**

- 第1版の動画の1コマです: トンネル越しに telnet でログインし、コマンドを いくつか打ち、ボード上の小さな Web ページを開きました。ボードは電源アダプタだけで 動いていて、相手は公式の Windows 版 WireGuard です。第1版でもデモには十分でした。 次に、これを本物の NuttX のドライバにしました。

**再生しない:** 静止画で十分。動画はライブデモの予備として取っておく。**これは apps 版。** この後のカーネル版の結果と混ぜて話さない。

## 25 — 目次 ⑤Version 2: the kernel driver (18:14 → 18:24)

**画面:** 目次と同じリスト。⑤Version 2: the kernel driver だけハイライト、他は薄い。

**話す(一言):** 第1版は以上です。第5部、難しい後半: 本物のカーネルドライバと、その確かめ方。

**間:** ハイライトされた行を指して一拍。

## 26 — なぜカーネルへ: アプリからドライバへ (18:24 → 19:12)

**画面:** 見出し「Version 1 proved the protocol. Version 2 makes it a real NuttX device.」。2枚の枠 — Version 1: an app(`apps/` にあり、カーネル内部(`psock_*`)に手を入れる。FLAT ビルドでしか動かず、正式な in-tree ドライバにはなれない)/ Version 2: a kernel driver(`drivers/net/wireguard/` に `wg0`、小さな `wg` コマンドが ioctl で設定。FLAT・PROTECTED・KERNEL のために作った)。下に3つのビルド モードの1行説明(FLAT: アプリとカーネルが1つのメモリ空間 / PROTECTED: MPU がアプリをカーネルの メモリから締め出す / KERNEL: プログラムごとに自分のアドレス空間(MMU))。

**話す:**

- なぜカーネルに移したのか。
- 第1版はアプリです。`apps` にあり、カーネルのソケットの内部に手を入れています。 すべてが1つのメモリ空間を共有する FLAT ビルドなら、それで動きます。でも正式な in-tree ドライバには決してなれず、アプリとカーネルが分けられる PROTECTED や KERNEL ビルドでは動きません。
- そこでデバイスを `drivers/net/wireguard` としてカーネルに移し、小さな `wg` コマンドが ioctl で設定する形にしました。第1版は、プロトコルが NuttX の上で動くことを示しました。 第2版は、NuttX そのものに合わせることです: どのビルドモードでも使える普通のデバイスで、 コミュニティがレビューできる形にすること。

## 27 — カーネル側の設計: 全体像 (19:12 → 19:58)

**画面:** 図。ユーザー空間には `wg` コマンドだけ(apps/system/wg、genkey・set・show)。ioctl(ポインタを 含まない平らな構造体1つ。秘密鍵は書き込み専用)がユーザー/カーネルの境界を越えて、カーネル内の `wg0`(drivers/net/wireguard)に届く。`wg0` は自分の UDP ソケットと受信スレッドを持ち、タイマー・ピア・ クッキーを管理し、NuttX の暗号(crypto/)を使う。左に NuttX のネットワークスタック、右に暗号化された UDP(wlan0 / usrsock → ピア)。

**話す:**

- 実際のドライバの形です。
- ユーザー空間にあるのは、小さな `wg` コマンドだけです。これが ioctl でカーネルに 設定を送ります。ioctl が運ぶのは、中にポインタを含まない平らな構造体1つなので、 PROTECTED や KERNEL ビルドでも、カーネルがユーザーメモリへのポインタをたどる必要は ありません。秘密鍵は書き込み専用です: どの ioctl も、それを返しません。

- カーネルの中では、`wg0` は普通のネットワークデバイスです。UDP ソケットと受信スレッドを 自分で持っているので、第1版のスレッドとソケットのバグは、設計上もう起きません。暗号処理は NuttX 自身のものです。プロトコル本体は今も wireguard-lwip 由来の BSD-3-Clause のコードで、そのことは LICENSE ファイルに書いてあります。

## 28 — 落とし穴 5・6/6: もう2つのバグ、1つは NuttX 自身 (19:58 → 21:05) *(27と一体)*

**画面:** 見出し「Two more bugs, and one was in NuttX itself.」。左: NuttX 自身の暗号 (`chacha20poly1305` がカウンタをナンスの違うバイトに置く。カウンタ0では一致するので、ハンドシェイクは 通り、最初より後のデータパケットが失敗する。2行の修正、独立した `crypto:` の変更)。右: KERNEL ビルド だけで起きる(ピアの設定で 3KB のカーネルスタックに 6KB の配列 → あふれてパニック。sim では出ない。ヒープへ移した)。下に1行: SPRESENSE(cxd56)が NuttX 13.0.1 と master で `CONFIG_RTC_HIRES` を 有効にすると起動しない。手元で修正済み、報告は下書きのみで未送信。

**話す:**

- カーネルへ移す途中で、さらに2つのバグが出ました。そのうち1つは NuttX 自身のものでした。
- 1つ目は、NuttX 自身の ChaCha20-Poly1305 です。カウンタをナンスの違うバイトに 書き込んでいました。カウンタが0のときは両方の並びが同じなので、ハンドシェイクは 通り、その後、最初より後のデータパケットが復号に失敗します。NuttX の中でこのコードを 呼ぶものが今までなかったので、誰も踏んでいませんでした。修正は2行で、独立した crypto の変更として分けてあります。
- 2つ目は、カーネルビルドでしか起きないクラッシュです。ピアの設定が、3キロバイトの カーネルスタックに6キロバイトの配列を置いていました。スタックが大きい sim では一度も 出ませんでした。バグ4と同じスタックの教訓が、一段深いところで起きたものです。 配列はヒープに移しました。
- もう1つ、ボード側から: NuttX 13.0.1 と master で、高分解能 RTC のオプションを有効にすると SPRESENSE が起動しませんでした。原因を見つけて手元で直しました。報告は下書きまでで、まだ送っていません。

## 29 — 検証: どこで何が動いたか (21:05 → 22:48) *(核心・削らない)*

**画面:** 見出し「Real boards in FLAT builds. KERNEL and PROTECTED on QEMU.」。カード3枚 — FLAT・実機 (ESP32-S3: 2026-10-12、提出するシリーズそのもの。Linux とハンドシェイク、両方向の ping、最大 MTU まで / ESP32-S3 + SPRESENSE: 以前の版で Windows クライアントとトンネル / SPRESENSE: 40回の down/up でメモリは同じ、受信スレッドは 6096 中 1472B)、KERNEL・QEMU rv-virt(Linux WireGuard とトンネル、 `wg` はシステムコールで呼ぶ別プログラム / 4 CPU: 通信しながら設定変更と down/up、例外なし、1回の スクリプト実行)、PROTECTED・QEMU rv-virt(トンネルは動く、すべての ioctl が MPU の境界を越える / まれな例外: `wg` のスタックあふれ、システムコールが呼び出し側のスタックで動くため / 既定値を大きくして 修正、20/20)。下に「Not shown yet: PROTECTED or KERNEL on real hardware」(CoreS3 で PROTECTED を試したが、ボードがブートローダ経由のイメージを一切起動しない。WireGuard の問題ではない)。GitHub への案内(QRコード付き)。

**話す:**

- どの版がどこで動いたか、正確に話します。ビルドモードは3つあり、分けて話します。
- FLAT、実機です。10月12日に、upstream 用に用意したシリーズそのものを、スタックちゃんの中の ESP32-S3 で動かしました: Linux の WireGuard とハンドシェイクし、両方向の ping が、最大のパケットサイズまで通りました。それより前のドライバの版は、両方のボードで、実 Wi-Fi 越しに Windows クライアントと動きました。SPRESENSE では メモリを測りました: 40回の down/up の後、まったく同じでした。受信スレッドが使ったのは、6キロバイトの既定値に対して1472バイト。この既定値はかつては推測でした。今は実測値です。

- KERNEL、QEMU です。`wg` コマンドはシステムコールでドライバに届く別のプログラムで、トンネルは Linux まで届きます。4 CPU では、設定を変えながら大量に流しました。例外は ありませんでしたが、これはスクリプト化した1回の実行で、レース条件をくまなく探した ものではありません。
- PROTECTED、これも QEMU です。トンネルは動き、すべての ioctl がメモリの境界を越えます。 繰り返すと、ときどき例外が起きました。原因は `wg` コマンドのスタックあふれです。この モードでは、システムコールが呼び出し側のスタックの上で動くからです。既定値を大きくして 直りました: 20回中20回成功。
- まだ見せていないのは、実機での PROTECTED と KERNEL です。CoreS3 で PROTECTED を試しましたが、このボードはブートローダ経由のイメージを一切起動せず、これは WireGuard の問題では ありません。第1版のアプリも、両方のボードで動きました。記録は GitHub にあります。

## 30 — 検証: ping が見逃すもの (22:48 → 23:46) *(核心・削らない)*

**画面:** 見出し「Three problems that ping would never notice.」。カード3枚 — 起動のたびに同じ鍵? (SPRESENSE: 3回のリセットで3つの違う鍵。簡単な確認で、乱数の質の証明ではない)/ down の後に鍵が メモリに残る?(sim: セッション鍵は down の後すべてゼロ。静的鍵は up をやり直すために意図的に残す)/ 数時間後に出るリーク?(sim: 151回の再鍵化、100回の down/up、何も失われない。メモリは起動時の数に戻る。 1週間の試験ではない)。

**話す:**

- このスライドは、動くトンネルが教えてくれないことです: ping では決して気づかない3つの問題。
- まず鍵です。NuttX は `/dev/urandom` の seed をビルド時の設定から取ることができます。そうするとボードは起動のたびに同じ秘密鍵を作り、それでもすべて動きます。そこで SPRESENSE を3回リセットし、3つの違う鍵を得ました。簡単な確認であって、乱数の質の 証明ではありません。
- 2つ目、シミュレータでデバイスのメモリを読みました。down の後、セッション鍵は すべてゼロでした。静的鍵は、もう一度 up できるように、意図的に残しています。
- 3つ目、長い試験で、これもシミュレータです: 151回の強制的な再鍵化と、100回の down/up。バッファは1つも失われず、メモリは起動時とちょうど同じ数に戻りました。 1週間動いたとか、シミュレータがボードと同じように振る舞う、とは言っていません。

## 31 — 設計判断: タイムスタンプ問題には楽な答えがない (23:46 → 25:09) *(核心・削らない)*

**画面:** 規則(再起動の後でも、相手が見たものより新しいタイムスタンプが要る)。Chosen(リアルタイム クロック + この起動で使った最大値。Linux・wireguard-go と同じ規則 + 時計が未設定なら警告)/ Not chosen (毎回保存: flash が摩耗 / flash に範囲を予約: 確かめられなかった書き込み保証が要る / 待つ: 前回動いて いた時間だけ)。実測のペア(時計未設定 → 75秒でハンドシェイクなし、時計設定 → 4.1秒)。

**話す:**

- ここは試験ではなく、判断が必要だった唯一の場所です。そして、その判断を測りました。
- WireGuard のリプレイ防御には規則があります: すべてのハンドシェイクは、相手が これまでに見たものより新しいタイムスタンプを持たなければならない — 再起動の 後でもです。デバイスは普通ハンドシェイクを始める側で、しかもバッテリー付きの 時計を持っていない可能性が最も高い側です。
- 毎回のタイムスタンプを保存すると flash が摩耗します。flash に範囲を予約するのが 長期的には正しい答えですが、それには、これらのファイルシステムで確かめられなかった 書き込みの保証が必要です。そしてただ待つなら、1か月

動いていたデバイスは、1か月 再接続できません。ここでは、楽な選択と正しい選択が一致しませんでした *(ダンブルドアの「正しいことと楽なこと」)*。

- そこで、リアルタイムクロックと、この起動の中で使った最大値を選びました — Linux と wireguard-go とまったく同じです。そして時計が一度も設定されていなければ、ドライバが 一度だけ警告を出します。
- それから実機で測りました。ボードには何も送りませんでした。相手からの ping は、ボードの代わりにハンドシェイクを始めてしまうからです。起動時のままの時計: 75秒 見ていてもハンドシェイクなし。時計を設定すると: 4.1秒。だからドライバは警告を出します。

## 32 — 目次 ⑥Live demo (25:09 → 25:19)

**画面:** 目次と同じリスト。⑥Live demo だけハイライト、他は薄い。

**話す(一言):** ここまでは技術の話でした。第6部: 実際に動くところを見てもらいます。

**間:** ハイライトされた行を指して一拍。

## 33 — デモ構成: スマホのネットワーク上の1本のトンネル (25:19 → 26:17)

**画面:** 構成図。会場 Wi-Fi(機器どうしの通信を遮断)→ スマホのテザリング (独自の小さなローカルネットワーク)→ ラップトップ(Docker 内の WireGuard ピア、wg0 10.10.0.1、音声ファイルを :8000 で配信)とスタックちゃん (ESP32-S3 + NuttX、テザリング上の wlan0 越しに wg0 10.10.0.2。apps 版 v0.1.1、FLAT ビルド)を 1本の WireGuard トンネルで結ぶ。SPRESENSE は点線の2台目のピア(wg0 10.11.0.2)。

**話す:**

- デモの構成です。
- ラップトップとスタックちゃんは、どちらも私のスマホのテザリングにつないで います。会場の Wi-Fi は使えますが、機器どうしの通信を遮断しているので、スマホに小さなローカルネットワークを作ってもらいます。スマホ自身の インターネットは会場 Wi-Fi でもモバイル回線でもよく、デモには必要ありません。
- ラップトップとスタックちゃんの間に WireGuard のトンネルが1本あります。ラップトップ側はコンテナ内の普通の Linux WireGuard で 10.10.0.1、スタックちゃんは 10.10.0.2 です。telnet も Web ページも、音声さえも、デモのすべてがこのトンネルを通ります。音声は事前にラップトップで作ったもので、クラウドの API はどこにも使っていません。
- 見てほしいのは2つです。ログインした後のアドレスと、パケットです。トンネルの外では、暗号化された UDP しか見えません。

## 34 — デモ (26:17 → 27:47)

**画面:** 7つの手順 — wg show / telnet 10.10.0.2 / ifconfig / stackchan face happy / stackchan say hello.wav / tcpdump の外と内 / もう一度 wg show。右側: 外 = 暗号化された UDP 51820 だけ、内 = telnet の文字列、「apps 版 (v0.1.1)、FLAT ビルド」の表示。

**話す:**

- このデモの要点は、特別なことが何も起きていない、ということです。スタックちゃんは、暗号化トンネルの向こう側にある普通のコンピュータです。このボードは apps 版で動いています — 動画と同じアプリです。カーネルドライバも このチップで動きましたが(試験の部で見たとおり)、ロボットのデモは apps 版の上に作っています。

- まずラップトップで wg show。ハンドシェイクが済んでいて、スマホの ネットワーク上のスタックちゃんのアドレスが見えます。次に 10.10.0.2 へ telnet します — これはトンネルの中のアドレスです。ifconfig を打つと両方が 見えます。スマホのネットワーク側の wlan0 と、トンネルの中の wg0。つまり、本当にトンネル越しにログインしています。
- これで操作できます。笑顔にして、hello と言わせてみます。声もラップトップから 同じトンネルを通って届いています。
- 最後にパケットです。左はスマホのネットワーク: 暗号化された UDP だけ。右はトンネルの中: 同じ telnet のやり取りが平文で見えます。もう一度 wg show すると、転送バイト数が増えています。Mischief managed(いたずら完了) *(忍びの地図の合言葉。デモの締めの一言として軽く)*。

**ライブ:** `scripts/stackchan/demo.sh` を使う。左の端末で `login`、右の2つの端末で `outside` と `inside`、前後に `show`。画面は左に端末、右にスタックちゃん(カメラか机の上)。

**本番前:** スマホのテザリングを ON(2.4 GHz / 互換性優先)、ラップトップを 接続、Docker Desktop を起動。スタックちゃんを `nsh_wifi.py` でつなぐ (パスフレーズはボードに保存しない)、`wg_setup.py` を1回実行。`stackchan say` を1回試す。テザリングを入れ直すとアドレス帯が変わることが あるので、そのときは `nsh_wifi.py` と `wg_setup.py` をやり直す。`say` の途中でボードが止まることがまれにある(約20回に1回)。止まったら 電源を入れ直して同じ手順。

**画面に出さない:** `wg showconf`(秘密鍵を表示する)、`%USERPROFILE%\stackchan-wg` の鍵ファイル、テザリングのパスフレーズ。

**フォールバック:** 録画。ライブが止まったらすぐ切り替える。

## 35 — 目次 ⑦Summary & status (27:47 → 27:57)

**画面:** 目次と同じリスト。⑦Summary & status だけハイライト、他は薄い。

**話す(一言):** 最後の部: 作業が今どこにあるか、学んだこと、そして持ち帰ってほしいこと。

**間:** ハイライトされた行を指して一拍。

## 36 — 還元する: 今の状態 (27:57 → 28:39) *(おまけ・最優先で削る)*

**画面:** 4項目 — 小さな `crypto:` の修正(ChaCha20-Poly1305 のナンス修正、それだけで役に立つので ドライバと分けた)/ ドライバ(`drivers/net/wireguard`: カーネルデバイス、ABI、文書、ネットワーク本体の 小さな変更2つ)/ コマンド(`apps/system/wg`: ドライバと話す。スタックの既定値は PROTECTED ビルドも カバーするようになった)/ ここまでの確認(checkpatch・nxstyle が通る、各コミットが単独でビルド、ライセンス一覧。シリーズは ESP32-S3 で動く(FLAT、2026-10-12))。下に小さく: 小さな PR として準備済み。まだ署名しておらず、upstream には何も送っていない。

**話す:**

- 作業が今どこにあるかです。
- 作業は小さな単位に分けてあります: それだけで役に立つ小さな暗号の修正、カーネルデバイスのドライバ、ユーザー空間の `wg` コマンド。PROTECTED のスタックの修正は、コマンドの既定値に入っています。どのコミットも patch の検査を通り、1つずつ単独で ビルドでき、シリーズそのものが ESP32-S3 の普通の FLAT ビルドで動いています。まだ何も送っていません。各コミットには、まだ私の署名と Assisted-by の行がありません — AI ツールに 手伝ってもらったので、この行が要ります。作業が本当はどこにあるかを正直に言い、そのうえで レビューに変えてもらう。それが Community over code です。

**「もうマージされたのか」と聞かれたら:** まだです — これは提出候補 であって、マージ可能という宣言ではありません。詳細は Q&A 資料参照。

**時間がなければ:** このスライドはまるごと飛ばしてよい。本編の主張 (検証・パターン)は背負っていない。飛ばしても質疑で聞かれたら答えられる ように、上の2点だけ頭に入れておく。

## 37 — 学んだこと (28:39 → 29:59)

**画面:** 見出し「Three things this project taught me.」。カード3枚 — Lesson 1: From application user to OS contributor(何年もアプリ側から NuttX を使ってきた / このプロジェクトでドライバ、カーネル、NuttX 自身のバグにまで踏み込んだ)/ Lesson 2(強調): AI helped me code, but understanding was my job(Claude を たくさん使い、調査もコードも速くなった / OS の内部は初めてだった。AI のコードをそのまま採らず、質問し、読み、試し、自分で説明できるまで続けた / それが大変だった)/ Lesson 3: A welcoming open-source community (充実した文書と貢献ガイド、活発な議論とレビュー / AI ツールに前向きで、責任は人間: `Assisted-by`、AI は `Signed-off-by` を付けない / CI やハッカソンが新しい人の参加を助ける)。下に Hope の1行。

**話す:**

- まとめの前に、学んだことを3つ。
- 1つ目。私はアプリケーションのユーザーとして NuttX に来ました。このプロジェクトで、ドライバ、カーネル、さらには NuttX 自身のバグにまで踏み込むことになりました。
- 2つ目。これははっきり言いたいことです。このプロジェクトでは AI、Claude をたくさん 使いました。調べものもコードを書くのも、ずっと速くなりました。でも、組み込み OS の 内部は私には初めてで、設計や動きを本当に理解するのは大変でした。AI が書いたコードを、そのまま採ることはできませんでした。質問し、NuttX のソースを読み、動きを試し、自分で 説明できるようになるまで続ける必要がありました。AI はコードを書くのを助けてくれましたが、理解するのは私の仕事でした。
- 3つ目はコミュニティです。NuttX には良い文書と貢献ガイドがあり、技術的な議論と レビューが活発です。AI ツールにも前向きですが、責任は人が持ちます: コミットには Assisted-by と書き、AI は Signed-off-by を付けません。そして CI やハッカソンが、新しい人の 参加を助けています。オープンソースは、コードを公開するだけのものではありません。新しい人が参加し、学び、貢献できるときに育つものです。
- これからも Apache NuttX とオープンソースのコミュニティに貢献していけたらと思っています。 そして NuttX がこれからも育っていくのを楽しみにしています。

**AI の話は飾らずに。** 告白でも自慢でもなく: 速くなった、それでも理解は大変で、それは自分の仕事だった。

## 38 — Summary: 作ったものと、技術の教訓3つ (29:59 → 30:47)

**画面:** Built: `wg0`、NuttX カーネルの中の WireGuard ネットワークデバイス。ESP32-S3 と SPRESENSE(FLAT)、QEMU 上の KERNEL / PROTECTED ビルドで、標準の WireGuard ピアと動く。小さな PR として準備済み、まだ何も 送っていない。3項目 — ハンドシェイクの先まで試す / きれいな OS の境界は報われる / 移植はプロジェクトを 良くしうる。

**話す:**

- まとめます。
- 作ったのは wg0、NuttX のカーネルの中の WireGuard ネットワークデバイスです。ESP32-S3 と SPRESENSE の普通の FLAT ビルドで、そして QEMU 上の KERNEL と PROTECTED ビルドで動きます。小さな PR として準備してあ

り、upstream にはまだ何も送っていません。
- 技術の教訓を3つ。1つ目、ハンドシェイクの先まで試すこと: 長時間のデータ、実機、そしてすべてのビルドモード。2つ目、自分のコードと OS の間にきれいな線を引くと報われます: apps 版は、x86、Cortex-A7、Xtensa、Cortex-M4F の4種類の CPU と、3つの NuttX のバージョンで、コード変更なしで動きました。3つ目、あるプロジェクトへの移植は、そのプロジェクトを 良くする機会になります: バグの1つは NuttX 自身の暗号処理にありました。

## 39 — 謝辞: この発表を支えてくれた人たち (30:47 → 31:22)

**画面:** 4つの枠 — Alan / Apache NuttX コミュニティ / Apache Software Foundation と CoC の運営・ボランティアの皆さん / Sony の NuttX 開発者(これまでの、そして今の)。

**話す:**

- 最後に、お礼を言わせてください。
- まず Alan。このカンファレンスを教えてくれて、発表を出すよう背中を押して くれました。次に Apache NuttX コミュニティ。GSoC の提案を書いたとき、 話を聞いて励ましてくれました。採択されなかった後も続けられたのは、その おかげです。3つ目に、Apache Software Foundation と、Community Over Code を 運営してくださっている皆さん。そして最後に、私より前の Sony のエンジニアの 皆さん。NuttX を製品に入れ、プロジェクトに多くを還元してきました。この仕事は、その上に立っています。

## 40 — ありがとうございました (31:22 → 31:32)

**画面:** GitHub リポジトリ `github.com/wwlapaki310/gsoc2026-nuttx-wireguard` と QR コード、デモ動画のリンク、Questions welcome。

**話す:**

- ありがとうございました。ご質問をお受けします — カーネルドライバ、バグ、ioctl ABI、ハードウェア、何についてでも。

**その後:** Q&A はスライドではなく coc-glasgow-qa.md を手元に。よくある質問(リベースの状況、#14、IOB 枯渇、checkpatch)は そこに答えがありますが、どのスライドにも載っていません。